

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework Assignment 3

- Due: **11:59 PM today (Monday, April 6)**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Make sure to **rename your notebook file** as instructed.

Project proposal and feedback

- Project proposal grades have been released and peer feedback is available on Lamaku.
 - Please review feedback from the instructor, TA, and your classmates available on Lamaku.
 - Incorporate suggestions where appropriate.
- Many projects are based on your personal or professional interests (or both).
 - Great to see strong engagement and relevance!
- Project titles and authors will be posted on Slack.

Final project

- You may incorporate additional methods as needed.
 - Time series analysis will be covered this and next week.
 - We'll also cover basic clustering methods after that.
- Ensure methods are:
 - Appropriate for your research questions.
 - Clearly explained and justified.

Final report

- The 3-4 page limit includes references.
 - ~ 450 words per page
 - Up to 2 additional pages for figures, tables, and any supplementary materials
- Results don't need to be perfect or as expected. Focus on:
 - Clear description of motivation, datasets, and overall project design
 - Clear methodology
 - Thoughtful interpretation
 - Insightful discussion

GitHub repository

- Create a GitHub repository for your project.
 - Make it public or grant access to the instructor and TAs if it's private.
- Upload code and any necessary documentation for running your code.
- Check data source policies.
 - If your dataset can be shared, include it in your repository.
 - If it cannot be shared, provide link(s) to the original data source instead.

Lecture 20

- Linear Regression
 - [statsmodels](#)
 - Multiple linear regression
- Non-Linear Regression
 - Nearest neighbor regression
 - Step functions (Piecewise constant functions)
 - Polynomial regression

Statsmodels

- This information can be found using the `statsmodels` library.
 - We use ordinary least squares model to build the regression model.
 - Ordinary least squares (OLS) is a method for estimating the unknown parameters in a linear regression model.
 - Estimates the parameters in a regression model by minimizing the sum of the squared residuals.
- Used the statistical language R's notation to describe the model.
 - Here “~” can be interpreted as “a function of” or “depends on”.

Recall: Advertising data

- Dataset shows **product sales** (in thousands of units) and **advertising budgets** (in thousands of dollars) across TV, radio, and newspaper.

```
advertising = pd.read_csv('data/Advertising.csv', usecols=[1, 2, 3, 4])  
advertising.head()
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

predictors

response

```
linregress(advertising["TV"], advertising["sales"])
```

```
LinregressResult(slope=0.0475366404330197, intercept=7.032593549127702, rvalue=0.7822244248616063,  
pvalue=1.4673897001948012e-42, stderr=0.0026906071877968703, intercept_stderr=0.4578429402734785)
```

```
import statsmodels.formula.api as smf
```

```
est = smf.ols('sales ~ TV', advertising).fit()  
est.summary().tables[1]
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	7.0326	0.458	15.360	0.000	6.130	7.935
TV	0.0475	0.003	17.668	0.000	0.042	0.053

```
est.params
```

```
Intercept    7.032594  
TV           0.047537  
dtype: float64
```

response

predictor

95% CI

```
# compute predicted values given the estimated model parameters
```

```
est.predict(advertising["TV"])
```

```
0      17.970775
```

```
1       9.147974
```

```
2       7.850224
```

```
3      14.234395
```

```
4      15.627218
```

```
...
```

```
195     8.848493
```

```
196    11.510545
```

```
197    15.446579
```

```
198    20.513985
```

```
199    18.065848
```

```
Length: 200, dtype: float64
```

Interpreting the simple linear regression

$$\beta_0 = 7.0326$$

- The intercept is the expected mean value of y when $x = 0$.
 - If we spend \$0 TV advertising, then we should expect, on average, sales of 7.03 thousand units.
- The intercept has no intrinsic meaning when x never equals 0.

$$\beta_1 = 0.0475$$

- β_1 quantifies the association between that variable and the response.
 - The **average** effect on the response (y) of a one unit increase in x .
- Each \$1,000 spent on TV advertising is associated with an increase in sales of approximately 47.5 additional units.

Multiple linear regression

- How can we leverage other information to predict sales?
 - Naive solution: Build independent regression and combine results.
- Problems with the naive solution:
 - Not completely clear how to combine the data.
 - Min, mean, or median, etc.? Why?
 - Each feature ignores the effect of the other features.
 - Does not account for interactions.

```
est = smf.ols('sales ~ radio', advertising).fit()  
est.summary().tables[1]
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	9.3116	0.563	16.542	0.000	8.202	10.422
radio	0.2025	0.020	9.921	0.000	0.162	0.243

```
est = smf.ols('sales ~ newspaper', advertising).fit()  
est.summary().tables[1]
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	12.3514	0.621	19.876	0.000	11.126	13.577
newspaper	0.0547	0.017	3.300	0.001	0.022	0.087

Multiple linear regression (cont'd.)

- We re-write our linear regression equation to account for all variables

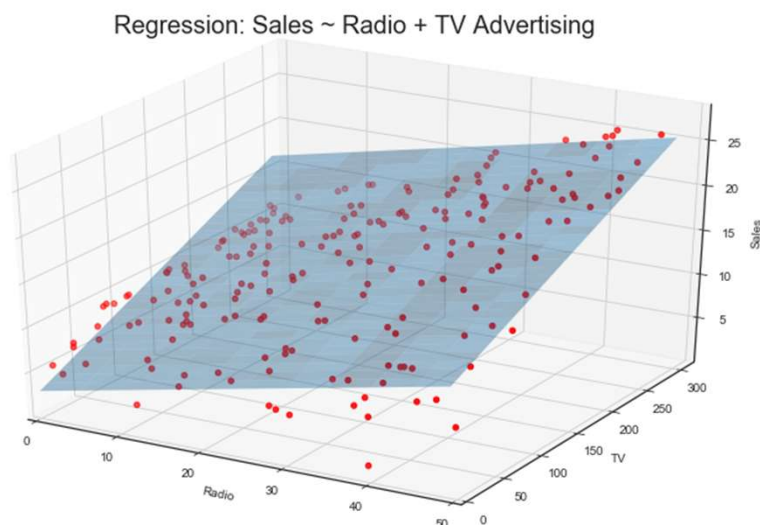
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

where:

- p is the number of predictors in the model.
- β_i is the average effect on y for a unit increase in the predictor x_i , holding all the other values constant.
- We can use statsmodels to compute the parameters of the new model:

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV_budget} + \beta_2 \times \text{radio_budget} + \beta_3 \times \text{newspaper_budget} + \epsilon$$

Estimating the regression coefficients



```
est = smf.ols('sales ~ TV + radio + newspaper', advertising).fit()
est.summary().tables[1]
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	2.9389	0.312	9.422	0.000	2.324	3.554
TV	0.0458	0.001	32.809	0.000	0.043	0.049
radio	0.1885	0.009	21.893	0.000	0.172	0.206
newspaper	-0.0010	0.006	-0.177	0.860	-0.013	0.011

Model interpretation

	coef	std err	t	P> t	[0.025	0.975]
Intercept	2.9389	0.312	9.422	0.000	2.324	3.554
TV	0.0458	0.001	32.809	0.000	0.043	0.049
radio	0.1885	0.009	21.893	0.000	0.172	0.206
newspaper	-0.0010	0.006	-0.177	0.860	-0.013	0.011

- For each additional \$1,000 in TV advertising, we expect, on average, an increase in sales of 45.8 units.
- For each additional \$1,000 in radio advertising, we expect, on average, an increase in sales of 188.5 units.
- The newspaper spending is not significant and does not contribute to the model.
 - Why is it different from when tested independently?

```
advertising.corr()
```

	TV	radio	newspaper	sales
TV	1.000000	0.054809	0.056648	0.782224
radio	0.054809	1.000000	0.354104	0.576223
newspaper	0.056648	0.354104	1.000000	0.228299
sales	0.782224	0.576223	0.228299	1.000000

Interaction

- Simple linear regression predicts an effect of 0.0547 (54.7 units) for newspaper.
- Multiple linear regression predicts an effect of ~ 0 .
- Recall that the correlation between radio and newspaper is ~ 0.35 .
 - Positive correlation indicates that in some cases, spending on radio advertising is accompanied by spending on newspaper advertising.
- What is happening?
 - radio advertising leads to increased sales.
 - radio and newspaper are correlated.
 - radio advertising is accompanied by newspaper advertising.
 - Part of the effect of newspaper advertising is actually accounted for by radio advertising.

Some important concepts to explore

- With dozens of predictors, manual analysis becomes impractical.
 - Which variables are related to each other?
 - R^2 increases as more predictors are added,
 - But this does **not** mean the model improves.
 - Often leads to **overfitting**.
- Selecting the best subset of variables is challenging.
 - See Chapter 3 of [Introduction to Statistical Learning](#).
- Some advanced models will automatically “shrink” the contribution of a feature to a model.
 - E.g.: ridge regression or lasso regression reduces the impact of “unnecessary” features.

Non-linear regression

- Complex models are rarely linear.
 - This is not to say that linear models are not used.
- Linearity assumption is often “good enough”, particularly for:
 - Quickly prototyping simple models that require full interpretability.
 - Tackling questions that are not well defined to benefit from more complex models.
- Always in search of better models but we need to always evaluate how much more leverage a more precise model is going to provide.

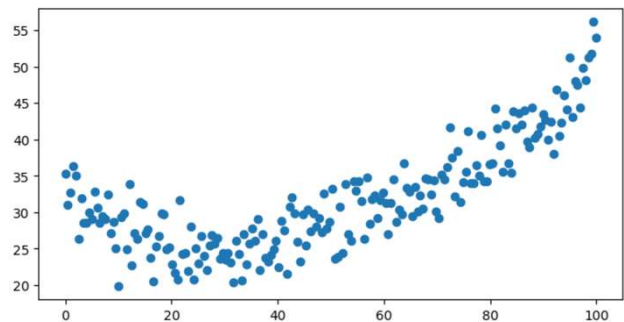
Non-linearity

- We will generate data from the following model:

$$y = 30 - 0.3x + 0.005x^2 + \varepsilon$$

```
errors = np.random.normal(0, 3, size=200)
x = np.linspace(0, 100, 200)
y = 30 + (-0.3 * x) + (0.005 * x ** 2) + errors

plt.figure(figsize=(8, 4))
plt.scatter(x, y);
```



```
from scipy.stats import linregress

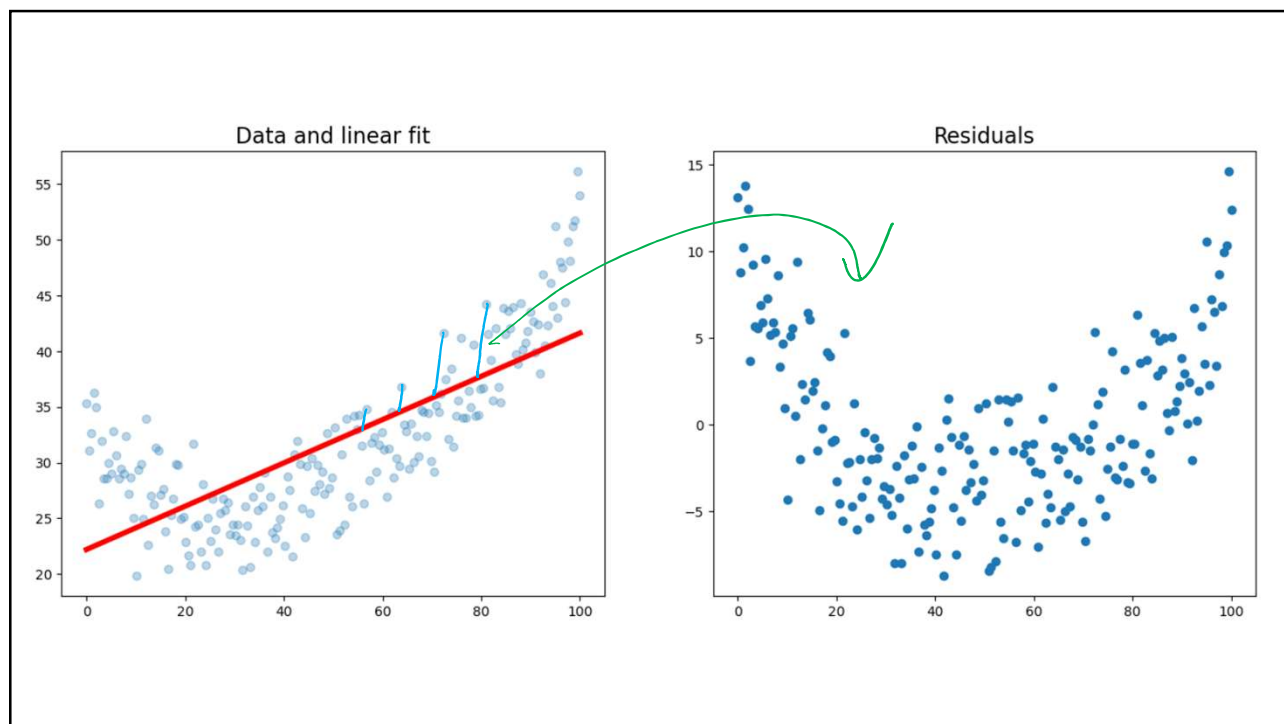
lm = linregress(x, y)

plt.figure(figsize=(16, 6))

plt.subplot(1, 2, 1)
plt.scatter(x, y, alpha=0.3)
plt.plot(x, lm.intercept + lm.slope * x, color='r', linewidth=4)
plt.title("Data and linear fit", fontsize=16)

Res_vals = []
for (x_i, y_i) in zip(x, y):
    y_hat = lm.intercept + lm.slope * x_i
    Res_vals.append(y_i - y_hat)

plt.subplot(1, 2, 2)
plt.scatter(x, Res_vals)
plt.title("Residuals", fontsize=16);
```



Non-linearity (cont'd.)

- True errors are normally distributed .
 - $\varepsilon \sim \mathcal{N}(\mu, \sigma)$ by construction.
- Issue arises after fitting an incorrect model.
 - Linear model omits the quadratic term.
 - Leads to model misspecification.
- Residuals are no longer purely random.
 - Contain systematic patterns (e.g., curvature).
 - Deviate from normality.

How to determine a non-linear model

- How do we handle the case where data is clearly non-linear?
- Within a sufficiently small region, the data will most likely be linear.
- For instance, we look at the region $x \in [35, 45]$.

```
positions = np.where((x >= 35) & (x <= 45))
x[positions]

array([35.1758794 , 35.67839196, 36.18090452, 36.68341709, 37.18592965,
       37.68844221, 38.19095477, 38.69346734, 39.1959799 , 39.69849246,
       40.20100503, 40.70351759, 41.20603015, 41.70854271, 42.21105528,
       42.71356784, 43.2160804 , 43.71859296, 44.22110553, 44.72361809])

y[positions]

array([27.82122032, 26.04816941, 29.10921996, 22.01886286, 26.96521285,
       23.74113045, 23.22306725, 24.14133288, 24.98817264, 26.13879981,
       22.524853 , 28.7753059 , 27.52486288, 21.5767188 , 30.71030594,
       31.99584156, 29.90966262, 25.90122446, 23.29894135, 29.74727983])
```

```
lm_2 = linregress(x[positions], y[positions])

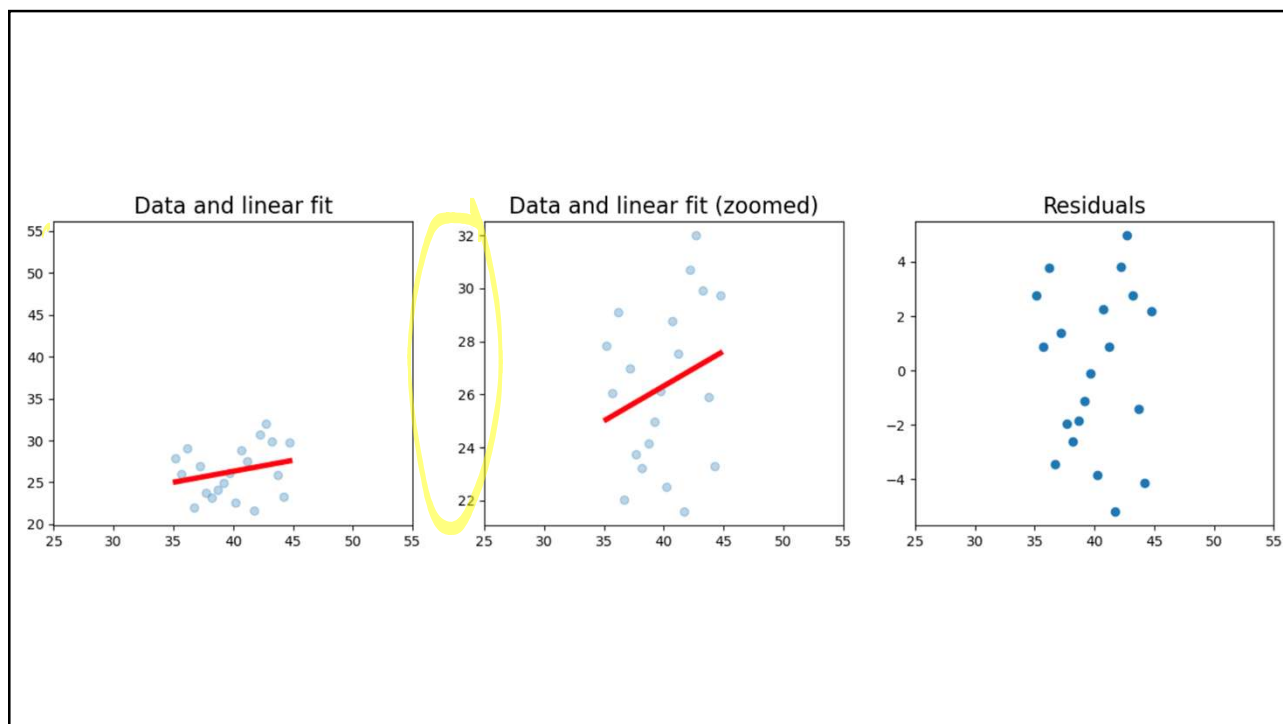
plt.figure(figsize=(16, 4))

plt.subplot(1, 3, 1)
plt.scatter(x[positions], y[positions], alpha=0.3)
plt.plot(x[positions], lm_2.intercept + lm_2.slope * x[positions], color='r', linewidth=4)
plt.ylim(y.min(), y.max())
plt.xlim(25, 55)
plt.title("Data and linear fit", fontsize=16)

plt.subplot(1, 3, 2)
plt.scatter(x[positions], y[positions], alpha=0.3)
plt.plot(x[positions], lm_2.intercept + lm_2.slope * x[positions], color='r', linewidth=4)
plt.xlim(25, 55)
plt.title("Data and linear fit (zoomed)", fontsize=16)

Res_vals = []
for (x_i, y_i) in zip(x[positions], y[positions]):
    y_hat = lm_2.intercept + lm_2.slope * x_i
    Res_vals.append(y_i - y_hat)

plt.subplot(1, 3, 3)
plt.scatter(x[positions], Res_vals)
plt.xlim(25, 55)
plt.title("Residuals", fontsize=16);
```



Nearest neighbor regression

- A naive approach is perhaps to compute the model as the average of some points above and below that point.
- For example, for $x = 40$, we take 3 observed data points immediately before and after 40, or any fixed number of nearest neighbors.
 - Say, for instance, we take 5 points.
- This is called a *nearest neighbor regression*.
- Here, let's just take the mean as a prediction within a small region.

```
x[130:150]
```

```
array([65.32663317, 65.82914573, 66.33165829, 66.83417085, 67.33668342,  
       67.83919598, 68.34170854, 68.84422111, 69.34673367, 69.84924623,  
       70.35175879, 70.85427136, 71.35678392, 71.85929648, 72.36180905,  
       72.86432161, 73.36683417, 73.86934673, 74.3718593 , 74.87437186])
```

```
np.searchsorted(x, 70)
```

```
140
```

```
x[np.searchsorted(x, 70)]
```

```
70.35175879396985
```

```
pos = np.searchsorted(x, 70)
```

```
print(x[pos: pos+5])
```

```
print(x[pos-5: pos])
```

```
[70.35175879 70.85427136 71.35678392 71.85929648 72.36180905]
```

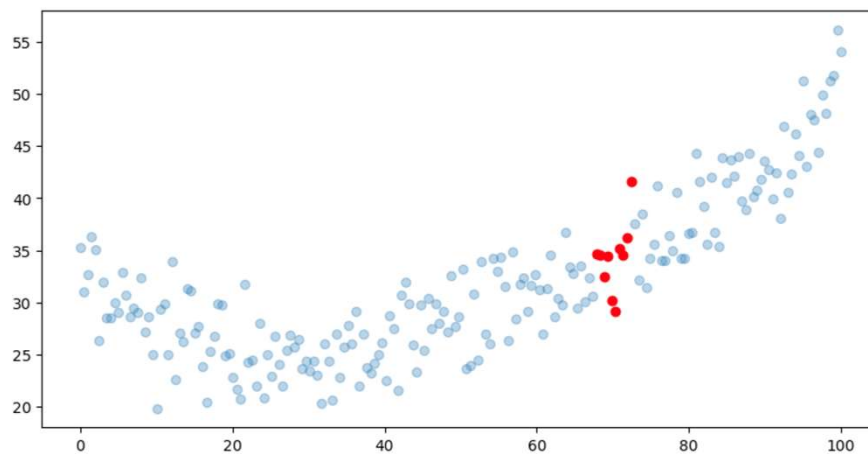
```
[67.83919598 68.34170854 68.84422111 69.34673367 69.84924623]
```

```
plt.figure(figsize=(12, 6))
```

```
neighbors = np.arange(pos-5, pos+5)
```

```
plt.scatter(x, y, alpha=0.3)
```

```
plt.scatter(x[neighbors], y[neighbors], color="red");
```

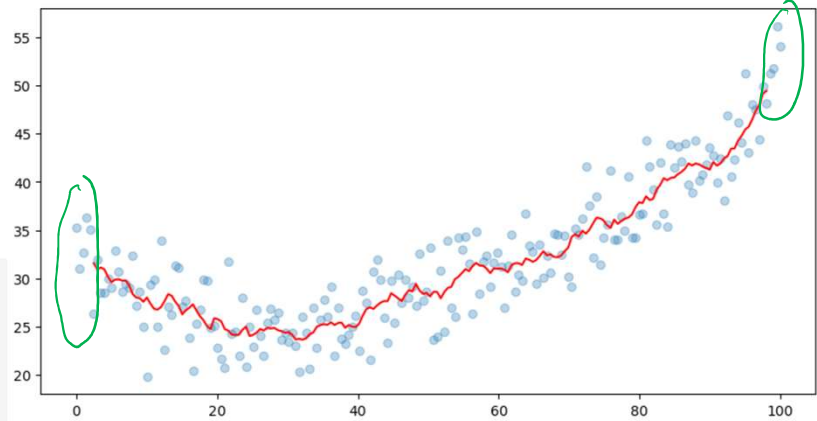


```
plt.figure(figsize=(12, 6))

reg_curve = []

for i in x[5:-4]:
    pos = np.searchsorted(x, i)
    neighbors = np.arange(pos-5, pos+5)
    reg_curve.append(y[neighbors].mean())

plt.scatter(x, y, alpha=0.3)
plt.plot(x[5:-4], reg_curve, color="red");
```



What are the issues with this approach?

Problems with nearest neighbor regression

- Computationally slow.
 - Distance calculations are required for all points to find neighbors.
 - Becomes expensive when the dataset is large or when there are many predictors.
- Highly affected by outliers.
 - Extreme values among nearest neighbors can skew predictions.
- Limited at data boundaries
 - Fewer neighbors at the edges make predictions unreliable or impossible.

Step functions

- We can remedy some of these shortcomings by discretizing the x -axis.
- Break the range of x into bins, and fit a different constant in each bin.
 - Such a group-specific constant can be mean as in nearest neighbors.
- This amounts to converting a continuous variable into an ordered categorical variable.
- This is called a step function.

```
intervals = np.split(np.arange(len(x)), 10)
intervals

[array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19]),
 array([20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,
        37, 38, 39]),
 array([40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56,
        57, 58, 59]),
 array([60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76,
        77, 78, 79]),
 array([80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96,
        97, 98, 99]),
 array([100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112,
        113, 114, 115, 116, 117, 118, 119]),
 array([120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132,
        133, 134, 135, 136, 137, 138, 139]),
 array([140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152,
        153, 154, 155, 156, 157, 158, 159]),
 array([160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172,
        173, 174, 175, 176, 177, 178, 179]),
 array([180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192,
        193, 194, 195, 196, 197, 198, 199])]
```

```
intervals[4]
```

```
array([80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96,  
       97, 98, 99])
```

```
x[intervals[4]]
```

```
array([40.20100503, 40.70351759, 41.20603015, 41.70854271, 42.21105528,  
       42.71356784, 43.2160804 , 43.71859296, 44.22110553, 44.72361809,  
       45.22613065, 45.72864322, 46.23115578, 46.73366834, 47.2361809 ,  
       47.73869347, 48.24120603, 48.74371859, 49.24623116, 49.74874372])
```

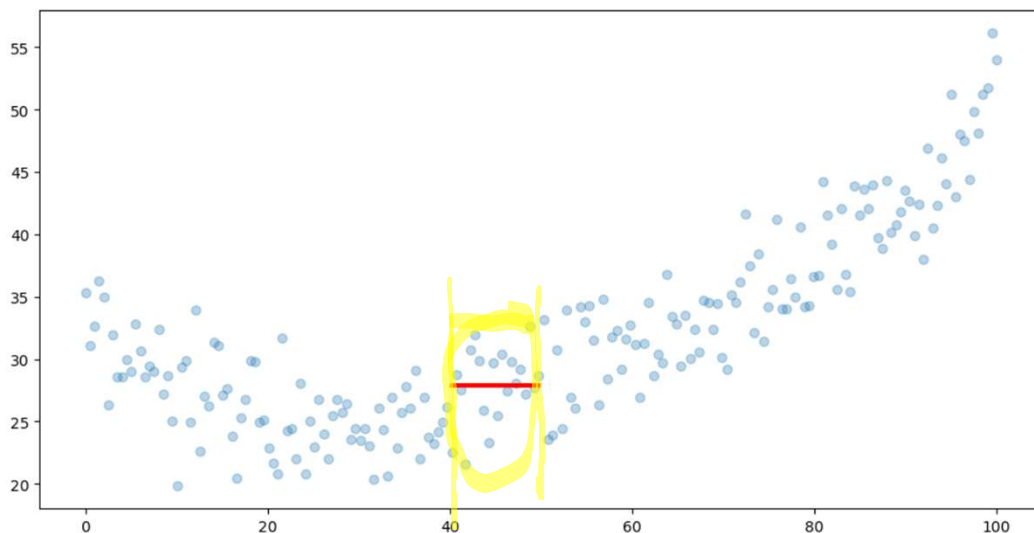
```
y[intervals[4]].mean()
```

```
27.92684321801365
```

```
plt.figure(figsize=(12, 6))
```

```
plt.scatter(x, y, alpha=0.3)
```

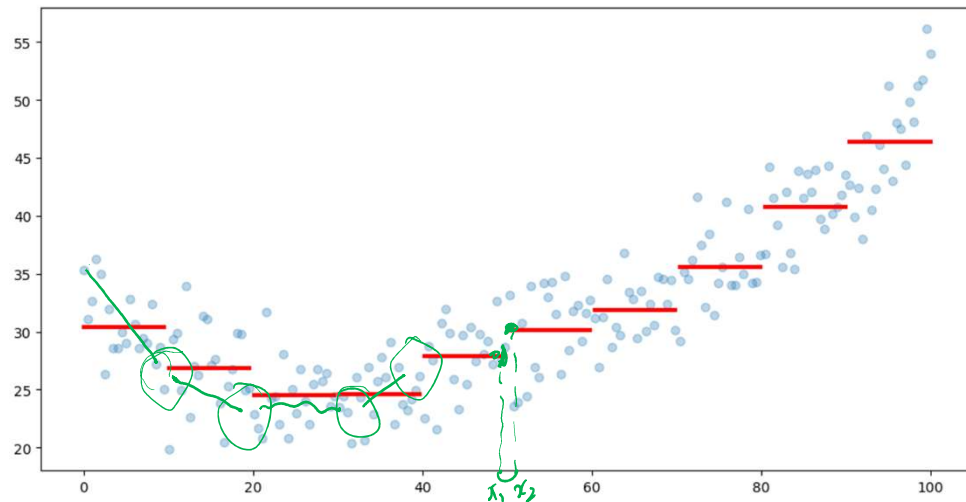
```
plt.plot(x[intervals[4]], [y[intervals[4]].mean()] * len(intervals[4]), color="r", linewidth=3);
```



```
plt.figure(figsize=(12, 6))

plt.scatter(x, y, alpha=0.3)

for i in range(len(intervals)):
    plt.plot(x[intervals[i]], [y[intervals[i]].mean()] * len(intervals[i]), color="r", linewidth=3)
```



Shortcomings of step functions

- How do you interpret sudden changes between two points close on the x -axis?
- Choice of cutpoints or "knots" can be problematic.
 - The step function would have been different if we had split the function into 11, 9, or 13 intervals.
 - Arbitrary knots can lead to substantial variation issues.
 - Different discrete intervals can lead to significantly different predictions.

Polynomial regression

- Rather than model the data with a linear first-degree polynomial, we will use a higher-degree polynomial.
 - E.g., second, third or even higher if needed.
- A first-degree polynomial:

$$y = \beta_0 + \beta_1 x$$

- A third-degree polynomial:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$

Using linear models with polynomial features

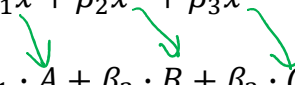
- Two-step process:
 1. Transform x into higher degree features.
 2. Fit the linear model using the transformed polynomial features.

Inferring polynomial features

- From the implementation point of view, this is just plain Ordinary Least Squares (OLS).
- Transform:

into:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$



$$y = \beta_0 \cdot 1 + \beta_1 \cdot A + \beta_2 \cdot B + \beta_3 \cdot C$$

such that $A = x$, $B = x^2$ and $C = x^3$.

- We can now consider A , B and C as new features of the model and use the same multiple linear regression.
 - The new representation of y is still considered a linear model.

Inferring polynomial features in scikit-learn

- The transform can be easily done using `PolynomialFeatures` in `scikit-learn` library.
 - Polynomial features are all polynomial combinations of the features with degree less than or equal to the specified degree.
 - Takes data as column vector.

Example

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix}$$

- For $x = [1, 2, 3, 4, 5]$
- In polynomial regression (third degree polynomial)

$$x' = (x^0 = 1, x^1 = A, x^2 = B, x^3 = C)$$

- Therefore,

$$X' = \begin{bmatrix} x^0 & x^1 & x^2 & x^3 \\ 1. & 1. & 1. & 1. \\ 1. & 2. & 4. & 8. \\ 1. & 3. & 9. & 27. \\ 1. & 4. & 16. & 64. \\ 1. & 5. & 25. & 125. \end{bmatrix}$$

```
x = np.array([1, 2, 3, 4, 5])
x.reshape(-1, 1)
```

```
array([[1],
       [2],
       [3],
       [4],
       [5]])
```

$$\begin{matrix} \text{cost: } x_1, x_2 \\ x_1, x_2 \\ x_1^2, x_2^2, x_1 x_2 \\ x_1^3, x_1 x_2^2, x_1^2 x_2, x_2^3 \end{matrix}$$

```
from sklearn.preprocessing import PolynomialFeatures
```

```
x = np.array([1, 2, 3, 4, 5])
```

```
poly = PolynomialFeatures(degree=3)
poly.fit_transform(x.reshape(-1, 1))
```

```
array([[ 1.,  1.,  1.,  1.],
       [ 1.,  2.,  4.,  8.],
       [ 1.,  3.,  9., 27.],
       [ 1.,  4., 16., 64.],
       [ 1.,  5., 25., 125.]])
```

fit()
transform()

calculate the
of output
features